In [1]:
```
import plotly.io as pio
pio.renderers.default = 'png'
```

Pandas Logo

Seaborn Logo

Plotly Logo

Altai

Pandas (https://pandas.pydata.org/docs/)

Seaborn (https://seaborn.pydata.org/api.html)

Plotly (https://plotly.com/python-api-reference/)

Altair (http viz.github.io/do

Introduction aux Diagrammes à Barres

Les diagrammes à barres sont un outil de visualisation fondamental utilisé pour afficher des données catégorielles avec des barres rectangulaires. La longueur de chaque barre est proportionnelle à la valeur qu'elle représente. Les diagrammes à barres sont excellents pour comparer des quantités entre différentes catégories.

Quand utiliser les diagrammes à barres :

- Comparaison de données catégorielles.
- Affichage de la fréquence ou du nombre de différentes catégories.
- Visualisation des tendances entre les catégories.

Terminologie de base :

- Barres : Les colonnes rectangulaires représentant les valeurs des données.
- Axes : Les lignes horizontales (axe des x) et verticales (axe des y) qui définissent le graphique.
- Étiquettes : Le texte qui décrit les catégories et les valeurs.
- Titre : La description générale du graphique.

Exercice : Analyse de Diamants avec les Bibliothèques de Visualisation

Avant de commencer cet exercice, veuillez lire la documentation des bibliothèques suivantes pour vous familiariser avec leurs fonctionnalités de base :

- Matplotlib : https://matplotlib.org/ (https://matplotlib.org/)
- Seaborn : https://seaborn.pydata.org/ (https://seaborn.pydata.org/)
- Plotly : https://plotly.com/python/ (https://plotly.com/python/)
- Altair : https://altair-viz.github.io/ (https://altair-viz.github.io/)

Vous disposez d'un ensemble de données contenant des informations sur divers diamants. Votre tâche est de réaliser l'analyse et les visualisations suivantes en utilisant Matplotlib, Seaborn, Plotly et Altair. Pour chaque visualisation, fournissez un bref commentaire sur ce que vous observez concernant les forces et les faiblesses de chaque bibliothèque pour cette tâche particulière.

1. **Chargement et Inspection des Données :**
   - Chargez l'ensemble de données à partir de l'URL fourni : `"https://raw.githubusercontent.com/TrainingByPackt/Interactive-Data-Visualization-with-Python/master/datasets/diamonds.csv"`
   - Affichez les 5 premières lignes de l'ensemble de données pour comprendre sa structure.
   - Imprimez les noms des colonnes et leurs types de données.
   - Affichez les 10 premières lignes de l'ensemble de données.
2. **Visualisation par Diagramme à Barres :**
   - **Matplotlib :**
     - Créez un diagramme à barres pour visualiser le nombre de diamants pour chaque catégorie de 'cut' en utilisant Matplotlib.
     - Personnalisez ce diagramme à barres en ajoutant des étiquettes appropriées pour l'axe des x et l'axe des y, et définissez un titre pour le graphique.
     - Commentez la syntaxe de Matplotlib, ses capacités de personnalisation et sa facilité d'utilisation pour cette tâche.
   - **Seaborn :**
     - Créez le même diagramme à barres (nombre de diamants par 'cut') en utilisant Seaborn.
     - Personnalisez le graphique comme vous l'avez fait avec Matplotlib (étiquettes, titre).
     - Comparez le code de Seaborn à celui de Matplotlib. Qu'est-ce que Seaborn fait différemment ou plus facilement ? Comment le style par défaut se compare-t-il ?
   - **Plotly :**
     - Créez un diagramme à barres interactif (nombre de diamants par 'cut') en utilisant Plotly.
     - Personnalisez le graphique avec des étiquettes et un titre. Explorez les fonctionnalités d'interactivité de Plotly (info-bulles, zoom).
     - Commentez l'interactivité de Plotly, sa syntaxe et en quoi elle diffère de Matplotlib et Seaborn.
   - **Altair :**
     - Créez un diagramme à barres (nombre de diamants par 'cut') en utilisant Altair.
     - Personnalisez le graphique avec des étiquettes et un titre.
     - Commentez la syntaxe déclarative d'Altair et comment elle se compare aux autres bibliothèques.
3. **Analyse Supplémentaire (Facultatif) :**
   - Explorez davantage les données en créant des diagrammes à barres pour d'autres variables catégorielles telles que 'color' ou 'clarity' dans n'importe laquelle des bibliothèques.

- Essayez de visualiser le prix moyen pour chaque 'cut' ou 'color' en utilisant les différentes bibliothèques et commentez la façon dont chaque bibliothèque gère ce type d'agrégation.

Conseils :

- Utilisez la bibliothèque pandas pour charger et manipuler les données.
- Utilisez les bibliothèques Matplotlib, Seaborn, Plotly et Altair pour créer les diagrammes à barres.
- Référez-vous au notebook pour des exemples de code et des explications sur les fonctions dont vous pourriez avoir besoin.

1 - Data loading and Inspection

In [2]:
```
import pandas as pd

diamonds_url = "https://raw.githubusercontent.com/TrainingByPackt/Interactive-Data-Visualization-with-Python/master/datasets/diamonds.csv"
diamonds_df = pd.read_csv(diamonds_url)
diamonds_df
```

Out[2]:

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...	...
53935	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

53940 rows × 10 columns

In [3]:
```
diamonds_df.head(10)
```

Out[3]:

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
5	0.24	Very Good	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
6	0.24	Very Good	I	VVS1	62.3	57.0	336	3.95	3.98	2.47
7	0.26	Very Good	H	SI1	61.9	55.0	337	4.07	4.11	2.53
8	0.22	Fair	E	VS2	65.1	61.0	337	3.87	3.78	2.49
9	0.23	Very Good	H	VS1	59.4	61.0	338	4.00	4.05	2.39

In [4]:
```
print(diamonds_df.dtypes)
diamonds_df.describe(include=object)
```

```
carat      float64
cut         object
color       object
clarity     object
depth      float64
table      float64
price        int64
x          float64
y          float64
z          float64
dtype: object
```

Out[4]:

	cut	color	clarity
count	53940	53940	53940
unique	5	7	8
top	Ideal	G	SI1
freq	21551	11292	13065

In [5]:
```
diamonds_df.head(10)
```

Out[5]:

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
5	0.24	Very Good	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
6	0.24	Very Good	I	VVS1	62.3	57.0	336	3.95	3.98	2.47
7	0.26	Very Good	H	SI1	61.9	55.0	337	4.07	4.11	2.53
8	0.22	Fair	E	VS2	65.1	61.0	337	3.87	3.78	2.49
9	0.23	Very Good	H	VS1	59.4	61.0	338	4.00	4.05	2.39

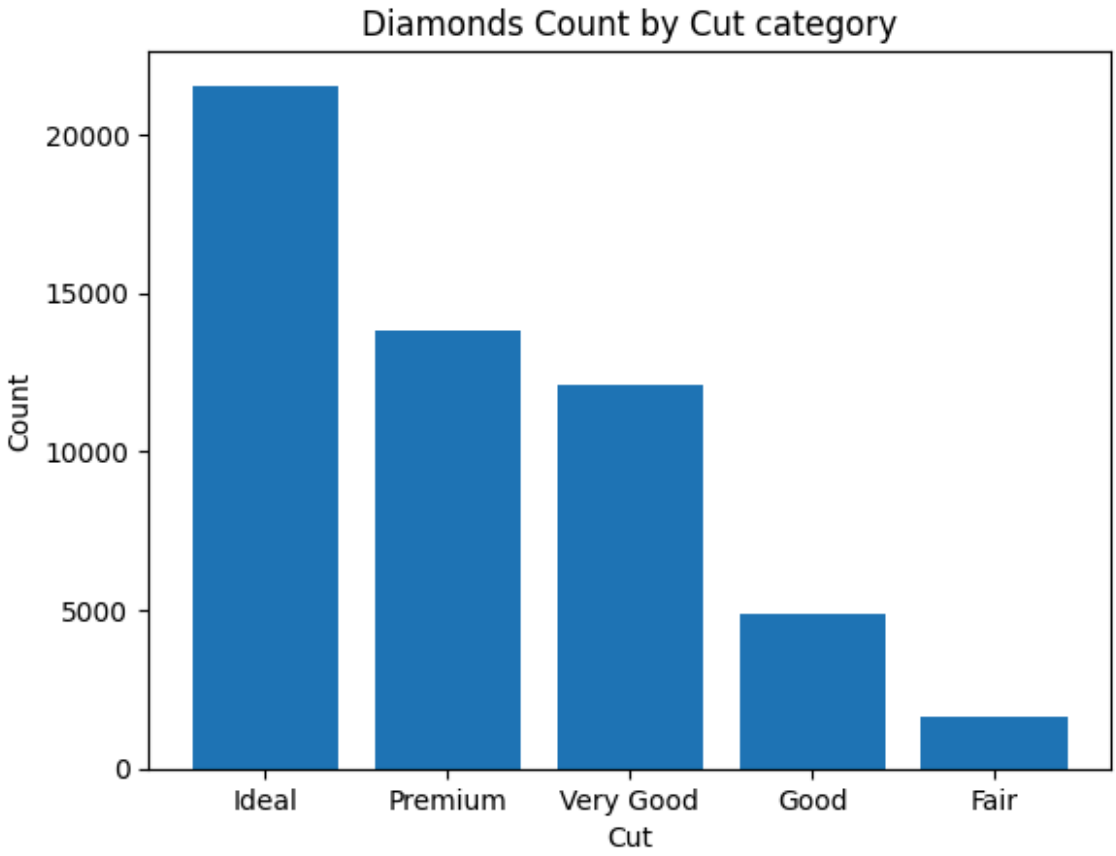
2 - Visualisation par Diagramme à Barres

Matplotlib

In [6]:
```
import matplotlib.pyplot as plt
import numpy as np
```

In [7]:
```
x = diamonds_df['cut'].value_counts().index # we set the x axis wit
h the cut categories (Ideal, Premium, Good, Very Good, Fair)
y = diamonds_df['cut'].value_counts().values # we set the y axis wi
th the counts of each cut category (number of diamonds per categor
y)
```

In [8]:
```
plt.bar(x, y) # we set the type of bar plot
plt.title("Diamonds Count by Cut category") # we set the title of t
he plot
plt.xlabel("Cut") # we set the label of the x axis
plt.ylabel("Count") # we set the label of the y axis
plt.show() # we display the histogram
```

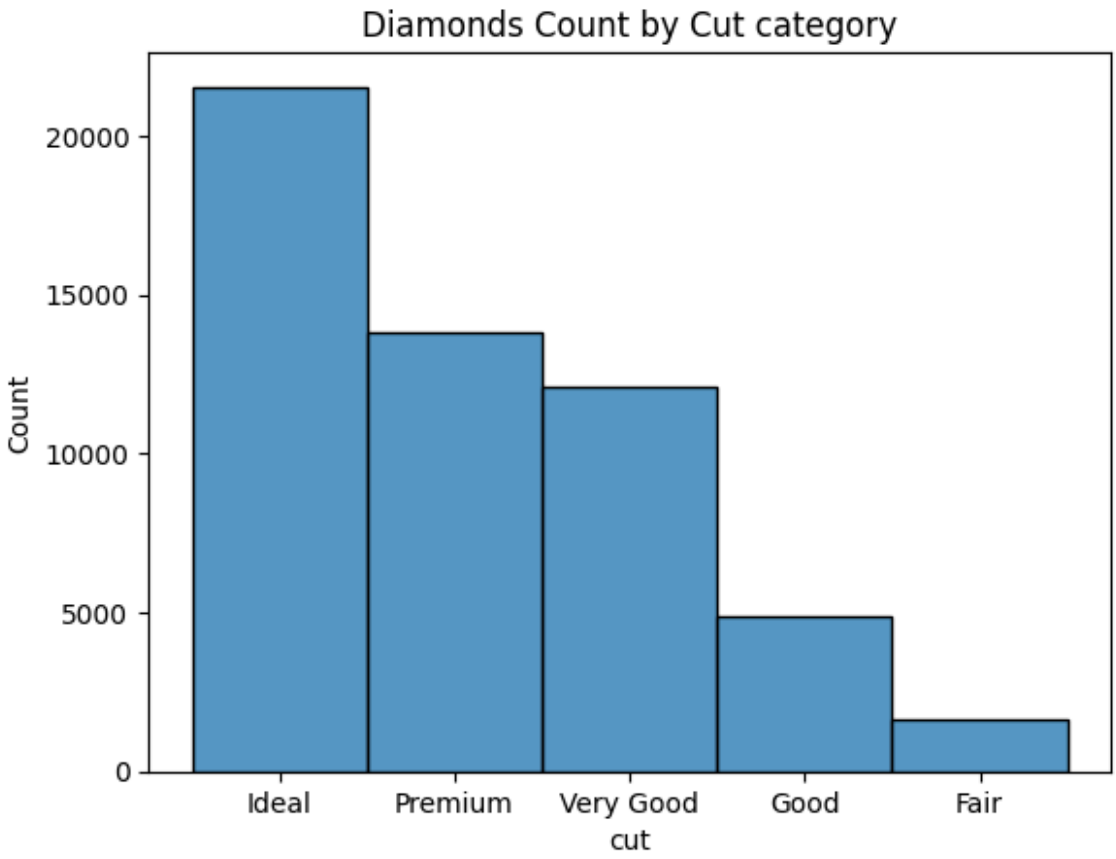


Seaborn

In [9]:
```
import seaborn as sns
diamonds = sns.load_dataset("diamonds")
```

In [10]:
```
cc = sns.histplot(diamonds, x="cut")
cc.set_title("Diamonds Count by Cut category")
```

Out[10]: Text(0.5, 1.0, 'Diamonds Count by Cut category')

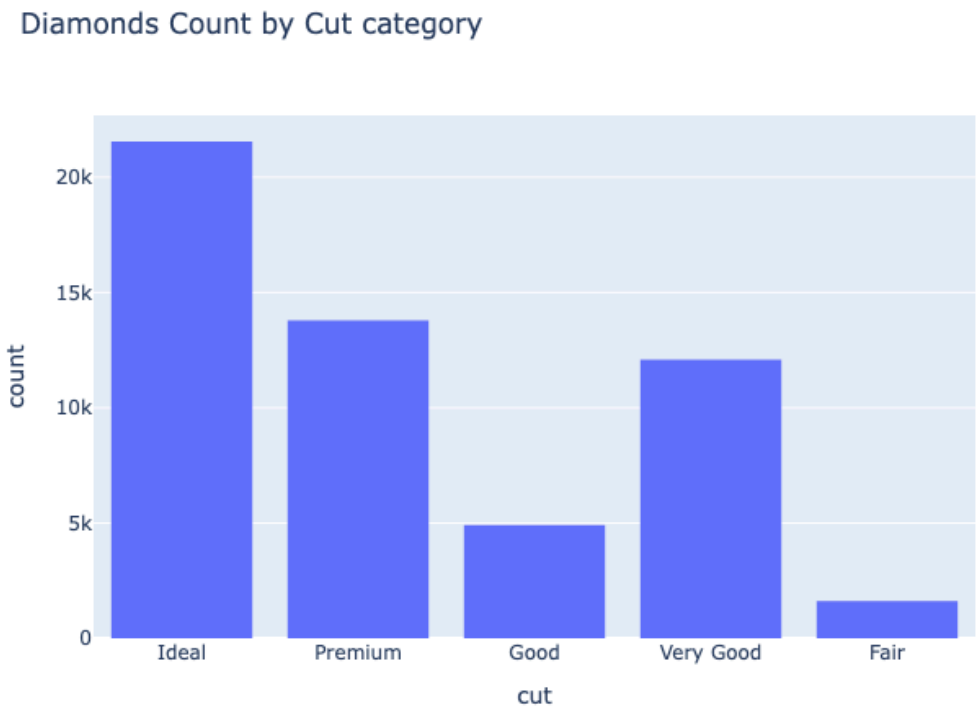


Easier than matplotlib to plot categorical data. It is automatically counting the values for each category and requires less code.

Plotly

In [11]:
```
import plotly.express as px
```

In [12]:
```
fig = px.histogram(diamonds, x='cut', title="Diamonds Count by Cut
category")
fig.show()
```



We can show exact values for each bar by hovering over them, we can move the graph around and zoom in/out easily. About the syntax, it is quite similar to seaborn but with added interactivity features. It seems easier to use than seaborn for basic plots.

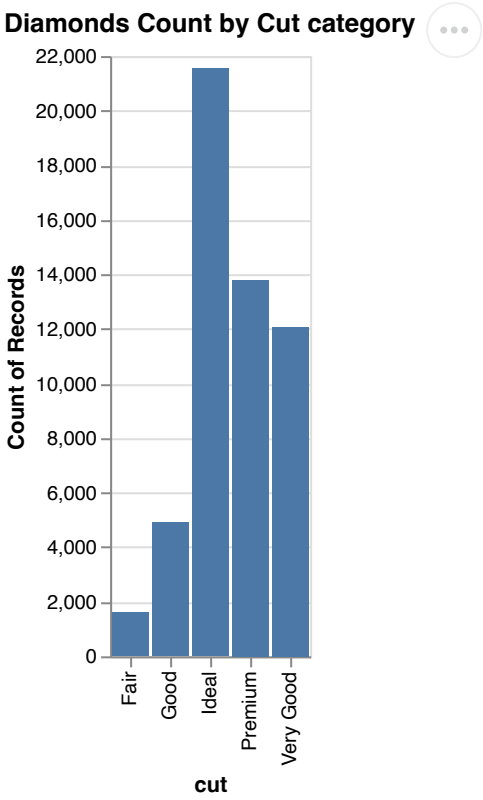
Altair

In [13]:
```
import altair as alt
alt.data_transformers.disable_max_rows() # for large dataset we nee
d this line
```

Out[13]:
```
DataTransformerRegistry.enable('default')
```

In [14]:
```
alt.Chart(diamonds).mark_bar().encode(x="cut", y='count()').propert
ies(title='Diamonds Count by Cut category')
```

Out[14]:



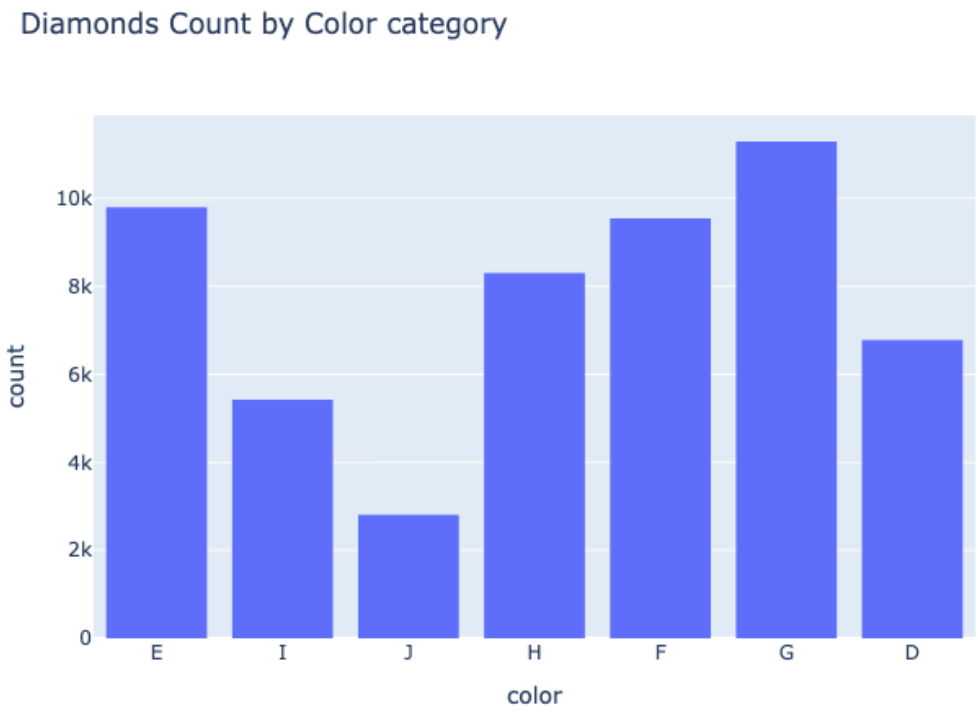
Altair seems to be more explicit than seaborn and plotly but less than matplotlib. It uses a declarative syntax which is different from the other libraries.

Exercice supplémentaire

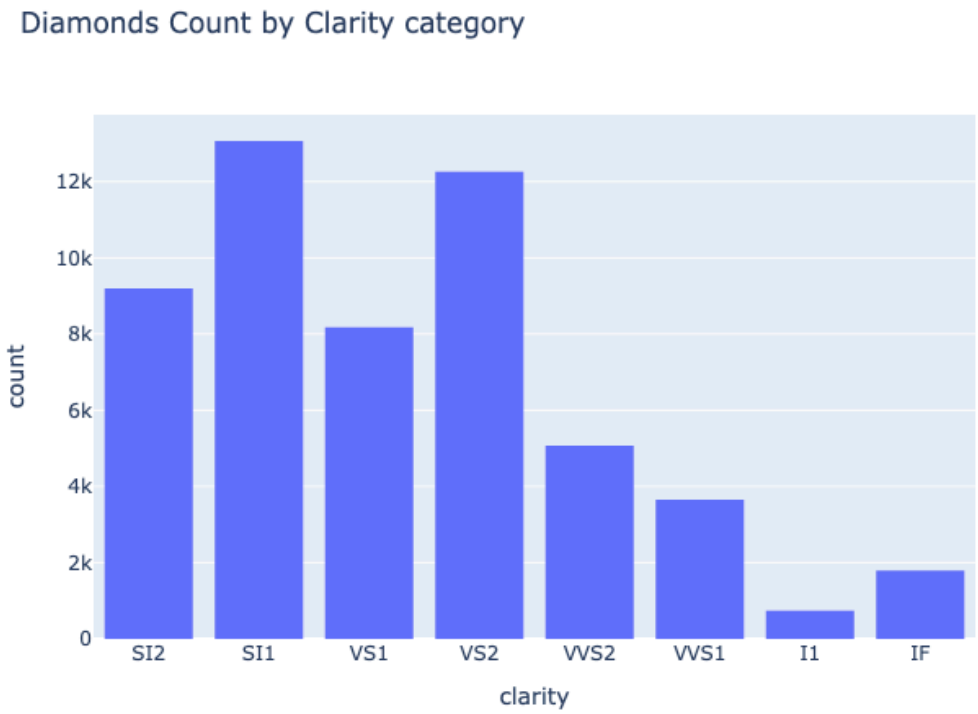
Plotly

Diamonds Count by Color and Clarity category

In [15]:
```
fig = px.histogram(diamonds, x='color', title="Diamonds Count by Co
lor category")
fig.show()
```



In [16]:
```
fig = px.histogram(diamonds, x='clarity', title="Diamonds Count by
Clarity category")
fig.show()
```



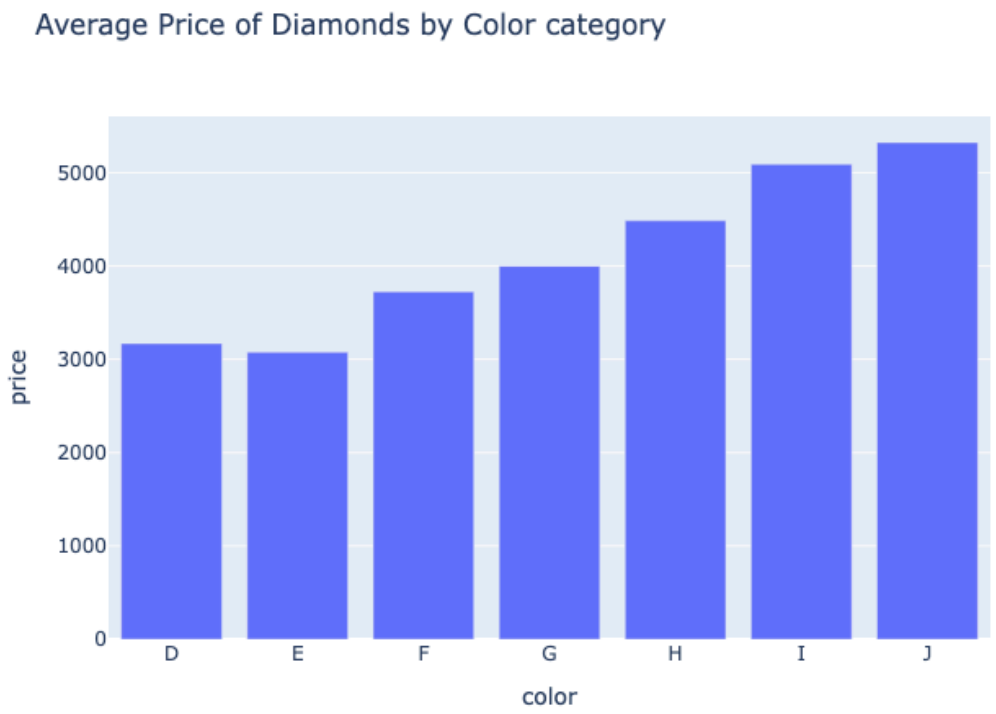
Visualize the average price of diamonds for each color category

In [17]:
```
prix_par_couleur = diamonds.groupby('color')['price'].mean().reset_
index()

fig = px.bar(prix_par_couleur, x='color', y='price', title="Average
Price of Diamonds by Color category")
fig.show()
```

```
/var/folders/0q/rj6cctzd35985_r1j_3p2x1w0000gn/T/ipykernel_97700/33
99023316.py:1: FutureWarning:

The default of observed=False is deprecated and will be changed to
True in a future version of pandas. Pass observed=False to retain c
urrent behavior or observed=True to adopt the future default and si
lence this warning.
```



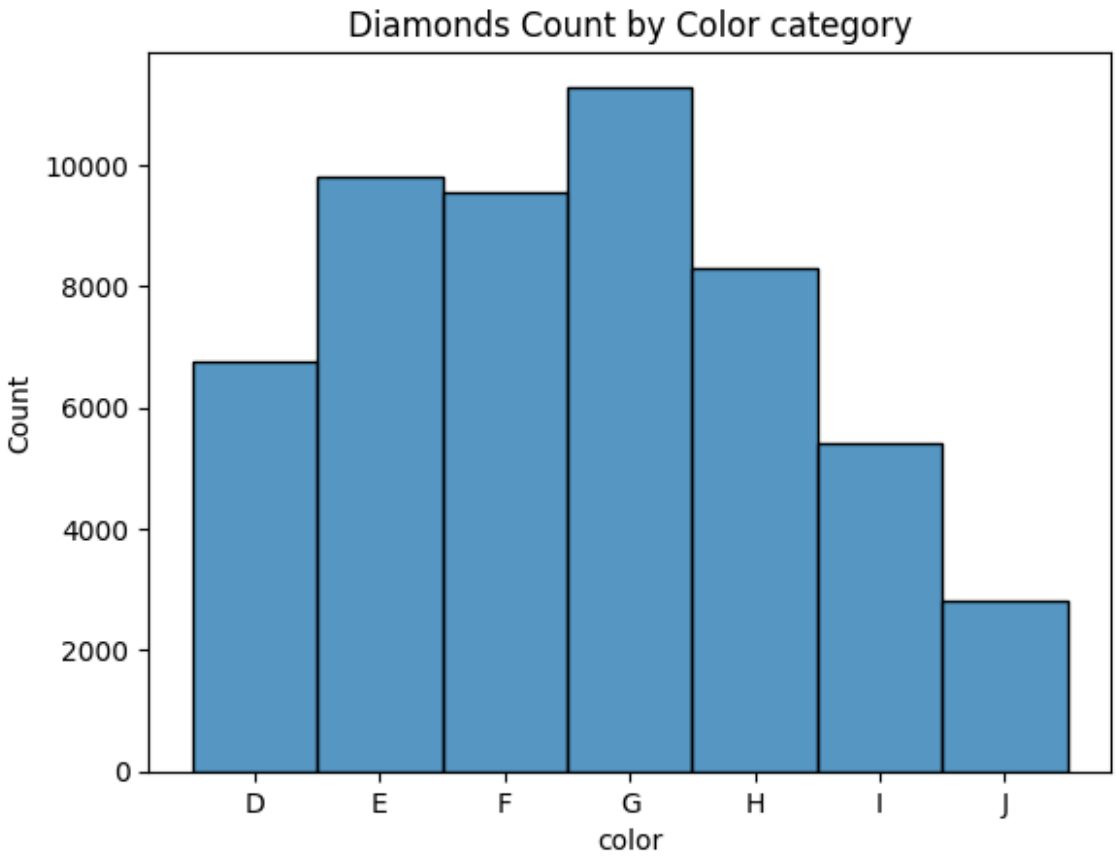
Plotly is very good for this type of visualization as it allows us to easily create interactive charts that can show more information when we hover over the bars. It also has a simple syntax for creating bar charts and can handle categorical data well. We need to use the `groupby` function to calculate the average price for each color category before plotting.

Seaborn

Diamonds Count by Color and Clarity category

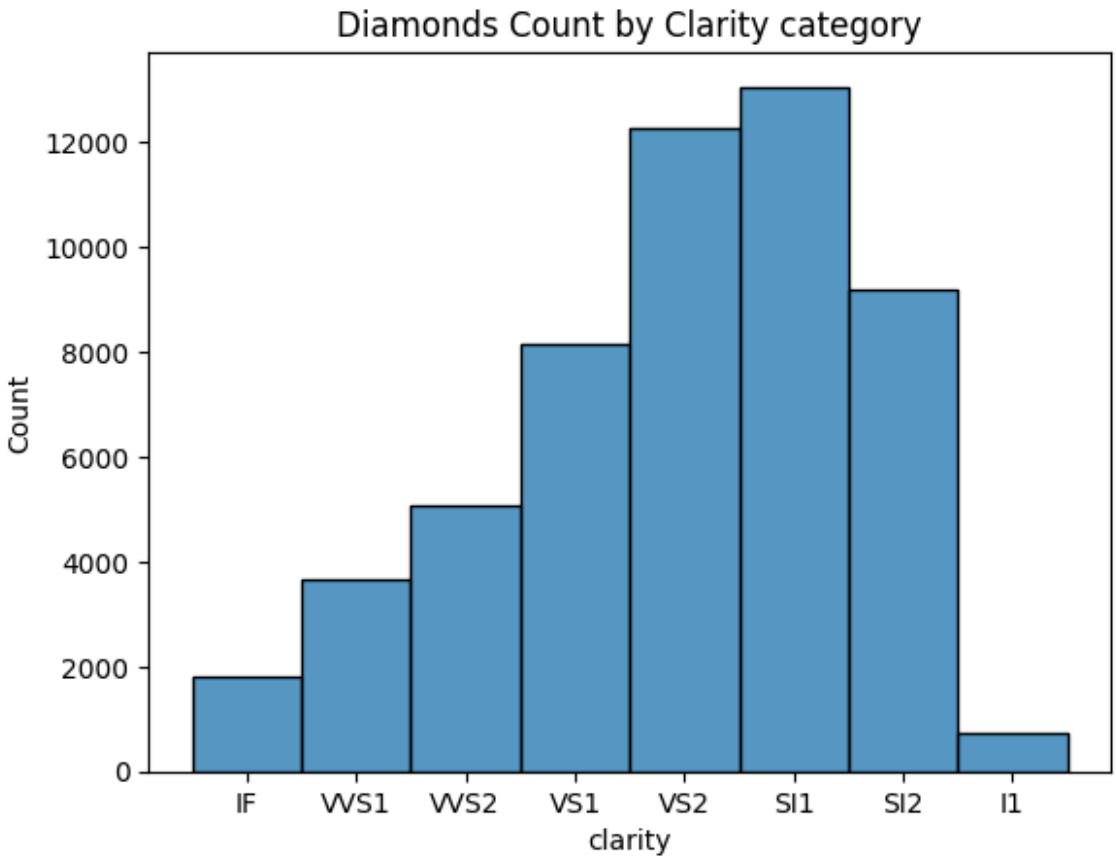
In [18]:
```
cc = sns.histplot(diamonds, x="color")
cc.set_title("Diamonds Count by Color category")
```

Out[18]:
```
Text(0.5, 1.0, 'Diamonds Count by Color category')
```



In [19]:
```
cc = sns.histplot(diamonds, x="clarity")
cc.set_title("Diamonds Count by Clarity category")
```

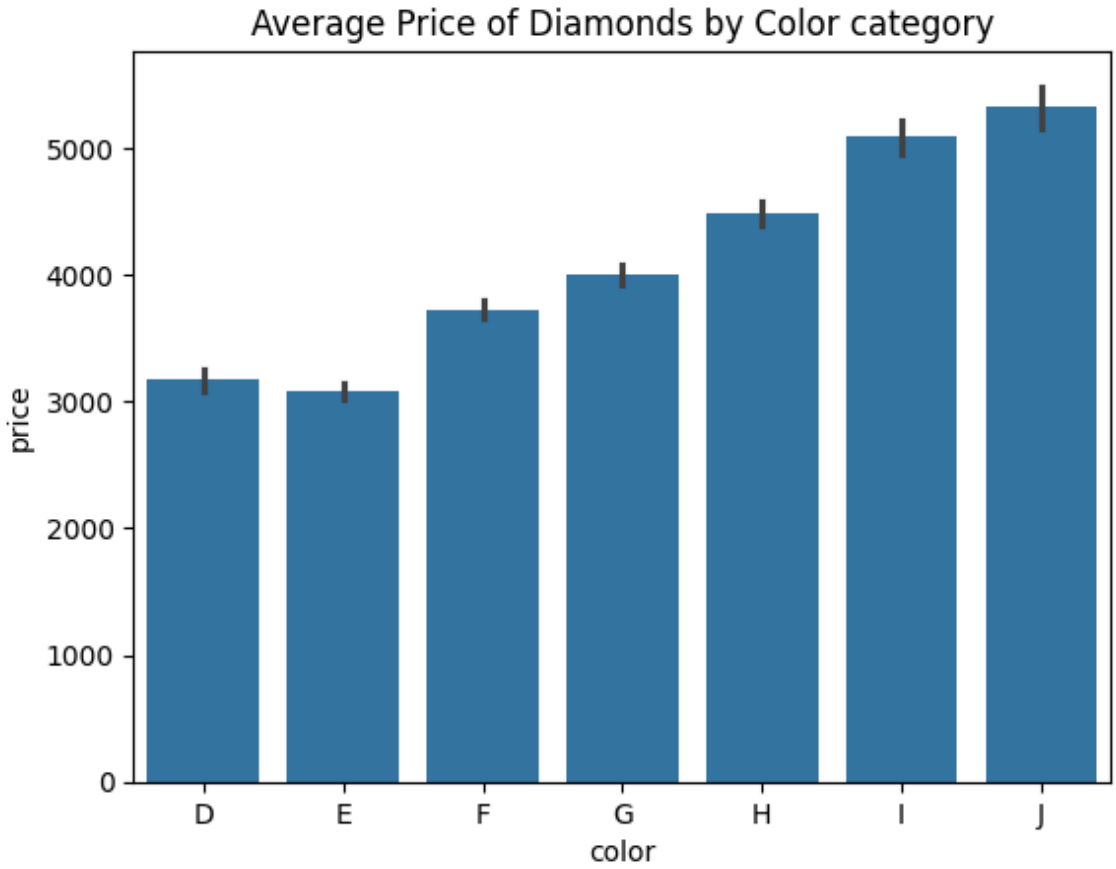
Out[19]: Text(0.5, 1.0, 'Diamonds Count by Clarity category')



Visualize the average price of diamonds for each color category

In [20]:
```
cc = sns.barplot(diamonds, x="color", y="price")
cc.set_title("Average Price of Diamonds by Color category")
```

Out[20]:
```
Text(0.5, 1.0, 'Average Price of Diamonds by Color category')
```



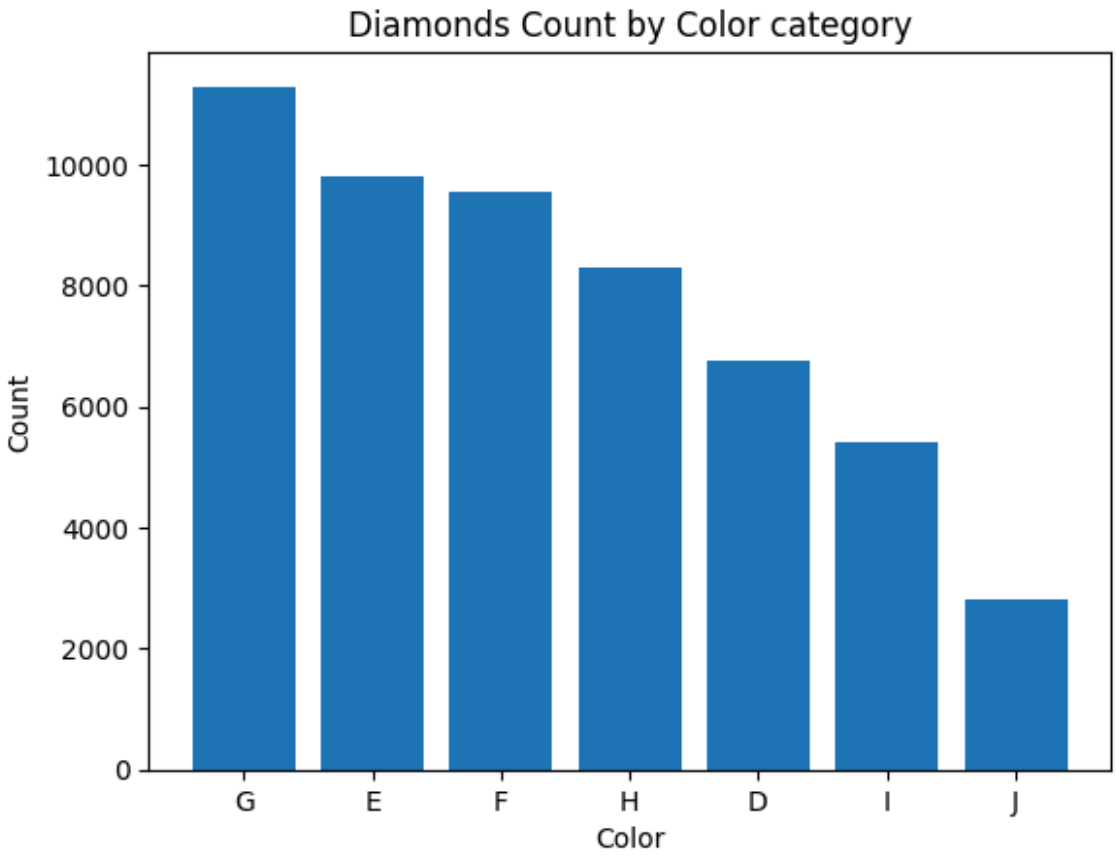
Seaborn is the same as matplotlib for this type of visualization, it is not as interactive as plotly but it has a simpler syntax and it is easier to customize the appearance of the plot. We also need to use the `groupby` function to calculate the average price for each color category before plotting.

Matplotlib

Diamonds Count by Color and Clarity category

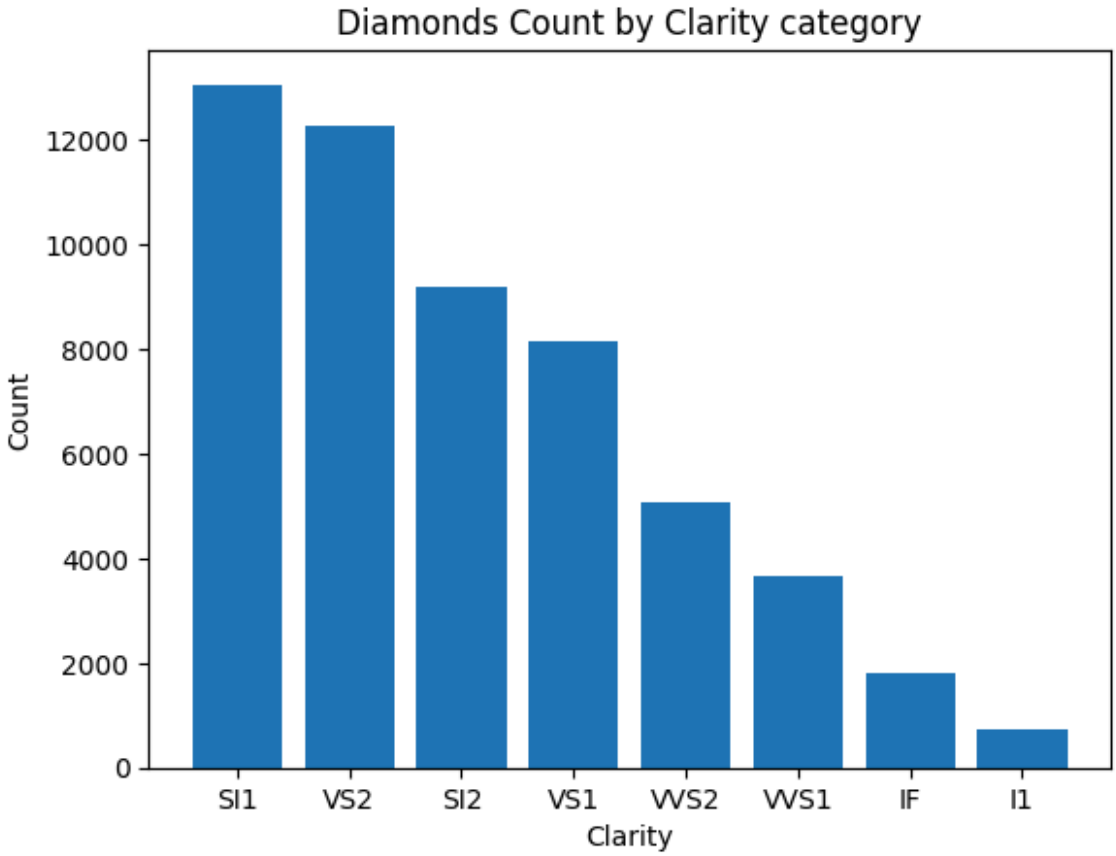
In [21]:
```
x = diamonds_df['color'].value_counts().index
y = diamonds_df['color'].value_counts().values

plt.bar(x, y)
plt.title("Diamonds Count by Color category")
plt.xlabel("Color")
plt.ylabel("Count")
plt.show()
```


Diamonds Count by Color category
Count
10000
8000
6000
4000
2000
0
G
E
F
H
D
I
J
Color

In [22]:
```
x = diamonds_df['clarity'].value_counts().index
y = diamonds_df['clarity'].value_counts().values

plt.bar(x, y)
plt.title("Diamonds Count by Clarity category")
plt.xlabel("Clarity")
plt.ylabel("Count")
plt.show()
```



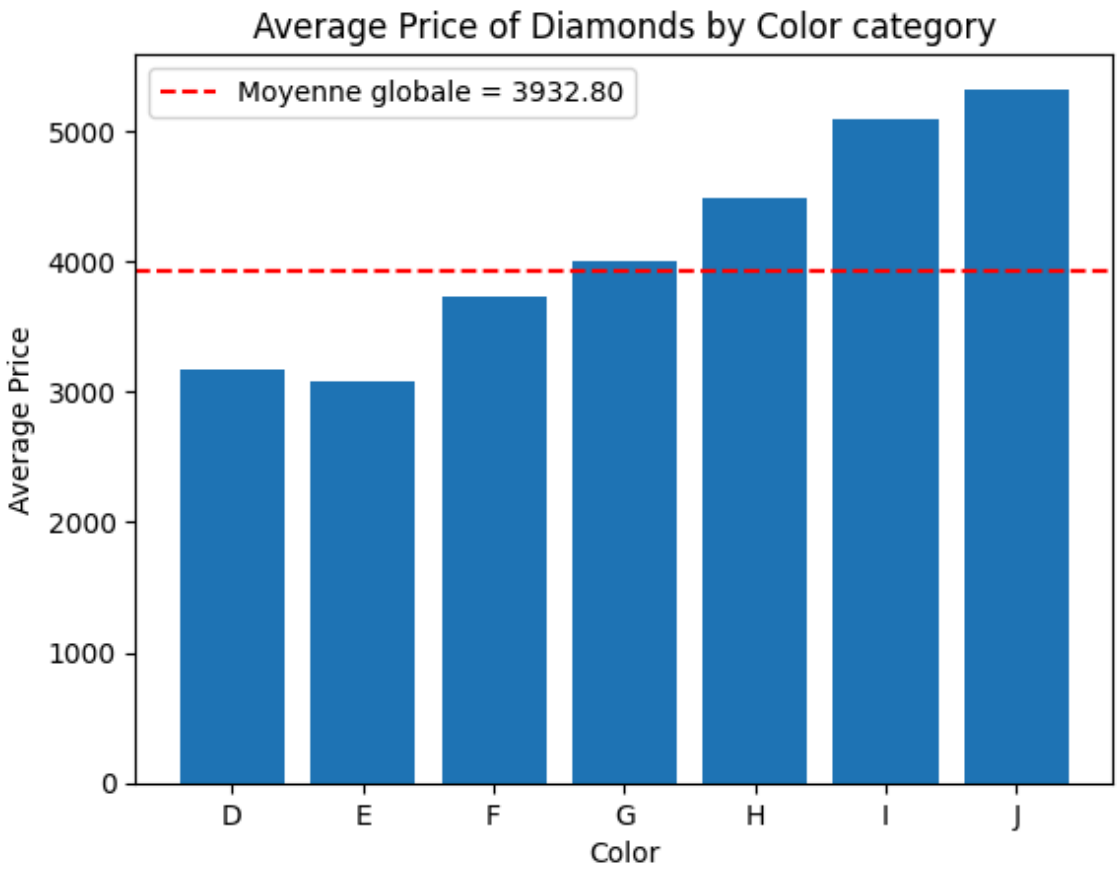
Visualize the average price of diamonds for each color category

In [23]:
```
moyenne_globale = np.mean(diamonds_df['price'])

prix_par_couleur = diamonds_df.groupby('color')['price'].mean()

x = prix_par_couleur.index
y = prix_par_couleur.values

plt.bar(x, y)
plt.axhline(y=moyenne_globale, color='red', linestyle='--', label=
f"Moyenne globale = {moyenne_globale:.2f}")
plt.title("Average Price of Diamonds by Color category")
plt.xlabel("Color")
plt.ylabel("Average Price")
plt.legend()
plt.show()
```



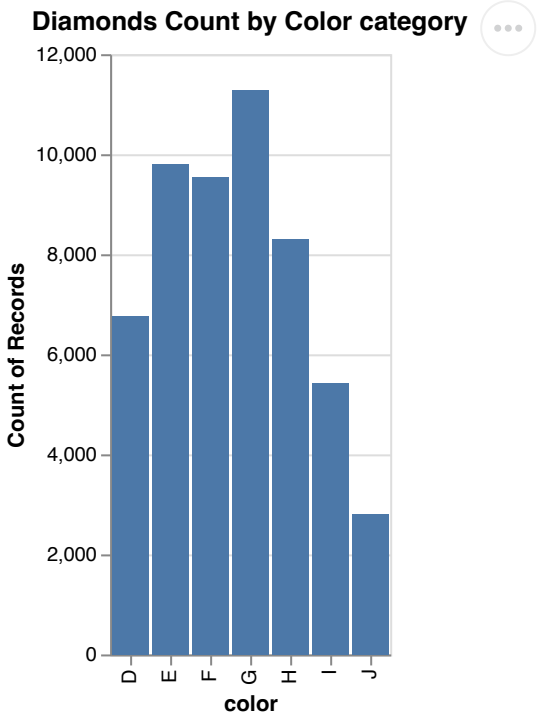
Matplotlib is the same as seaborn for the first part of the exercise, but for the second part, it requires more code to calculate the average price for each color category and to create the bar chart We need to use the `numpy` library and `groupby` function. It is less intuitive than seaborn for this type of visualization.

Altair

Diamonds Count by Color and Clarity category

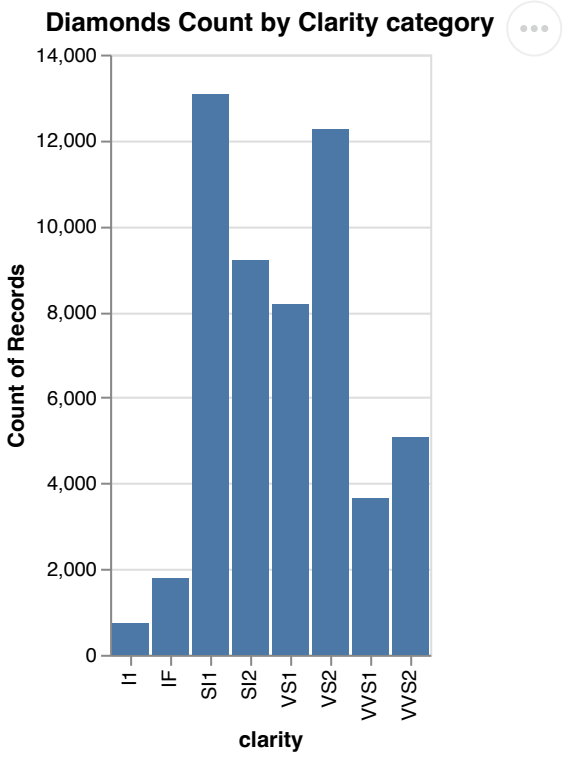
In [24]:
```
alt.Chart(diamonds).mark_bar().encode(x="color", y='count()').prope
rties(title='Diamonds Count by Color category')
```

Out[24]:



In [25]:
```
alt.Chart(diamonds).mark_bar().encode(x="clarity", y='count()').pro
perties(title='Diamonds Count by Clarity category')
```

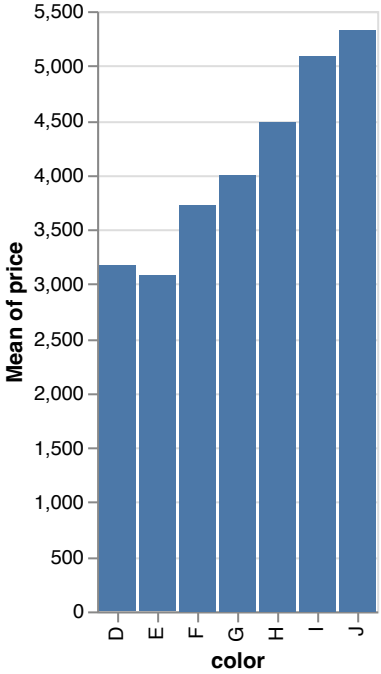
Out[25]:



Visualize the average price of diamonds for each color category

In [26]:
```
alt.Chart(diamonds).mark_bar().encode(x="color:N", y='mean(price):
Q').properties(title='Average Price of Diamonds by Color and Cut')
```

Out[26]:



Alatair is the easiest one and the most intuitive for this type of visualization, it has a simple syntax and it is easy to create interactive charts. It also has built-in support for handling categorical data and performing aggregations, which makes it very convenient for this task. We don't need to use the `groupby` function to calculate the average price for each color category, Altair can handle this directly in the plotting code.